

Mastering Advanced Policy Gradients & PPO

Complete Beginner-Friendly Breakdown: Policy Collapse, Kakade-Langford Bounds, Natural Gradients, TRPO, PPO-Clip & Massively Parallel Isaac Lab Simulation

Instructor: Prof. Sergey Levine
(UC Berkeley)

Companion: DEX-ROB Lab,
Tianjin University

Frameworks: Achiam (Spinning Up ch16) + Schulman & Levine TRPO/PPO

0. The "Mental Map": Why Does Lecture 10 Exist?

In Lecture 5 and Lecture 6, we derived the Policy Gradient Theorem and learned how to calculate policy updates. However, in standard gradient descent, taking a step that is even slightly too large can trigger a lethal disaster known as **Policy Collapse**.

In computer vision, taking a bad gradient step just means your loss temporarily increases on the current batch; the next batch re-centers the network. **In reinforcement learning, the policy generates its own future training data.** If one oversized gradient step makes the robot slam the knife violently, every rollout in the next iteration will result in crushed tomatoes and zero reward. The policy enters an unrecoverable death spiral.

Lecture 10 introduces Trust Region Policy Optimization (TRPO) and Proximal Policy Optimization (PPO): the algorithms that put a mathematical "safety leash" on policy updates, ensuring monotonic improvement and making deep RL robust enough to train complex robots in Isaac Lab.

1. Policy Collapse

Why Bad Steps
Permanently Crash RL

2. State Shift

$d^{\pi}(s)$ vs $d^{\pi_{old}}(s)$:
The Distribution Gap

3. TRPO & KL

Fisher Matrix &
Conjugate Gradients

4. PPO Clipping

$\min(rA, \text{clip}(r)A)$:
The Elegant Leash

5. Isaac Lab Speed

4096 Envs Parallel
Epoch Reuse

Part 1: The Catastrophe of Policy Collapse

(Slides 1–15, Spoken Transcript 02:10–16:40) Sergey Levine explains why policy optimization is uniquely fragile compared to standard supervised learning.

Supervised Learning (Stable)

Fixed Dataset (ImageNet / COCO)
Oversized Step → High Batch Loss
Next Batch: Recovers easily
Data distribution never changes!

Reinforcement Learning (FRAGILE)

Policy generates its own future data!
Oversized Step → Destructive Actions
Next Batch: 100% Crushed Tomatoes
CATASTROPHIC COLLAPSE (No Recovery)

1.1 The State Distribution Mismatch

When updating from $\pi_{\theta_{old}}$ to π_{θ} , we want to optimize:

$$J(\theta) = \mathbb{E}_{s \sim d^{\pi_{\theta}}(s)} \left[\mathbb{E}_{a \sim \pi_{\theta}(a|s)} [Q^{\pi_{\theta_{\text{old}}}}(s, a)] \right]$$

In standard policy gradients, we sampled states from the *old* distribution $d^{\pi_{\theta_{\text{old}}}}(s)$, pretending that changing the policy would not change which states the robot visits. Levine asks: *"When is it mathematically permissible to ignore this distribution change?"* **Answer: Only when the new policy π_{θ} stays strictly within a bounded 'trust region' around $\pi_{\theta_{\text{old}}}$!**

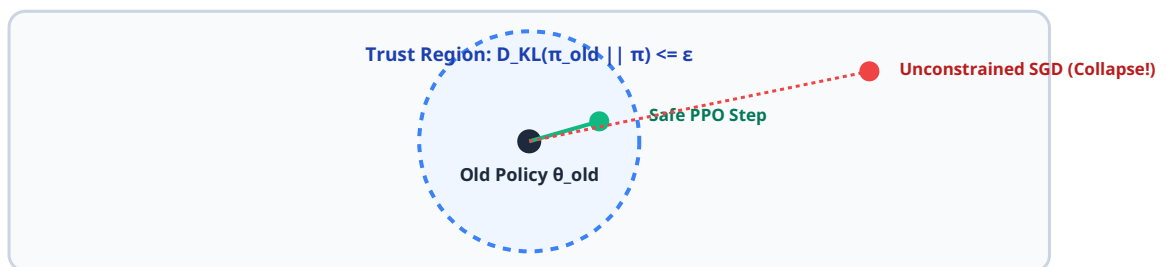
Part 2: Bounding Policy Shift: Trust Regions & KL Divergence

(Slides 16–35, Spoken Transcript 16:50–34:10) In their 2015 paper, Schulman, Levine et al. derived the fundamental theoretical bound for policy optimization:

2.1 The Kakade-Langford / TRPO Bound

$$J(\pi') \geq J(\pi) + \sum_s d^\pi(s) \sum_a \pi'(a|s) A^\pi(s, a) - C \cdot D_{\text{KL}}^{\max}(\pi, \pi')$$

Where $C = \frac{4\epsilon\gamma}{(1-\gamma)^2}$ is a constant depending on discount factor and maximum advantage, and $D_{\text{KL}}^{\max}$ is the maximum Kullback-Leibler (KL) divergence between policies across all states.



Part 3: From Natural Policy Gradients & TRPO to PPO

(Slides 36–52, Spoken Transcript 34:25–51:00) How did researchers originally enforce this KL constraint?

3.1 Natural Policy Gradient & TRPO

Natural Gradients measure distance in distribution space rather than parameter Euclidean space, using the **Fisher Information Matrix** F : $\Delta\theta \propto F^{-1} \nabla_{\theta} J(\theta)$. TRPO solves this via **Conjugate Gradient (CG)**. While mathematically principled, inverting F across 4,096 parallel environments is computationally prohibitive on GPU clusters.

Part 4: Proximal Policy Optimization (PPO) Deconstructed

(Slides 53–70 & Spinning Up ch16) In 2017, John Schulman, Sergey Levine et al. developed PPO to achieve TRPO's stability using first-order gradient descent.

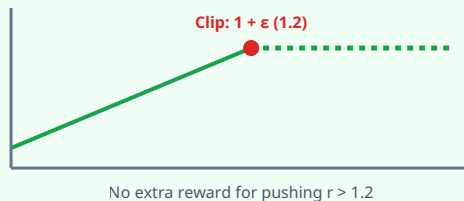
4.1 The Clipped Surrogate Objective

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$$

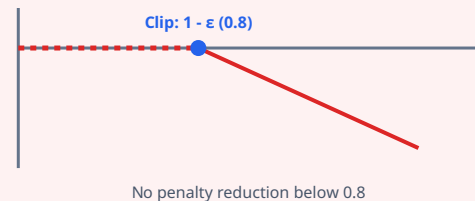
The PPO Clipped Objective:

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right]$$

Case 1: Advantage A > 0 (Good Action)



Case 2: Advantage A < 0 (Bad Action)



4.2 KL Early Stopping Heuristic (Achiam ch16)

Even with ratio clipping, running 5 to 8 epochs over a batch can cause the policy to drift excessively. In *Spinning Up in Deep RL*, Joshua Achiam documents the canonical early-stopping rule:

$$\text{If } \bar{D}_{\text{KL}}(\pi_{\text{old}} || \pi_\theta) > 1.5 \times d_{\text{target}} \implies \text{Terminate Epoch Loop Immediately!}$$

In your SkRL configuration, setting ``kl_threshold: 0.015`` enforces this exact safety brake.

SPINNING UP BUG ALERT: PREMATURE BATCH FLATTENING

In parallel environments, observations arrive as ``[T, num_envs, 33]``. **Never flatten the temporal dimension before computing GAE!** Advantages and returns must be calculated backwards through the temporal structure. If you flatten ``[T, num_envs]`` into ``[T * num_envs]`` before running GAE, the terminal step of environment 0 will erroneously bootstrap into the initial step of environment 1, corrupting credit assignment!

Part 5: Why Isaac Lab & Robotics Simulation Are Built for PPO

(Slides 71–85, Spoken Transcript 1:12:45–1:24:00) Why is PPO the undisputed default algorithm across modern GPU-accelerated robotics frameworks (NVIDIA Isaac Lab, Isaac Gym, Orbit)?

Feature	Standard RL (e.g., REINFORCE / SAC)	Isaac Lab + PPO Synergy
Data Reuse	On-policy REINFORCE must discard all rollouts after <i>one</i> gradient step.	4 to 8 Epochs per Rollout: Because clipping prevents policy collapse, PPO safely loops over the same simulation batch multiple times!
Parallel Scalability	Off-policy SAC struggles with 4,096 parallel streams because maintaining a 10M transition replay buffer exhausts GPU VRAM.	Direct On-Policy Streaming: Collect 64 steps across 4,096 parallel environments (262,144 transitions in 50 ms), optimize PPO, and flush buffer.
Optimizer Stability	TRPO requires complex second-order Conjugate Gradient solvers.	Standard first-order Adam optimizer with gradient clipping (1.0). Zero matrix inversions.

Part 6: Paper 1 Production Configuration: SkRL `ppo\_cfg.yaml`

```
# Production PPO Configuration for Dual-Arm Compliant Tomato Slicing agent:
class: PPO
rollouts: 64 # Steps collected per environment per iteration
learning_epochs: 5 # Passes over rollout buffer (safe due to clipping)
mini_batches: 8 # Sub-batches per epoch (32,768 transitions per mini-batch)
discount_factor: 0.99 #  $\gamma$ : Value horizon (~200 steps at 60 Hz)
lambda: 0.95 # GAE- $\lambda$ : Optimal contact bias/variance tradeoff
learning_rate: 3.0e-4 # Adam initial learning rate
learning_rate_scheduler: AdaptiveKL
learning_rate_scheduler_kwargs:
  kl_threshold: 0.015 # Dynamic safety brake: drops LR if policy drifts
ratio_clip: 0.2 # PPO trust region parameter  $\epsilon = 0.2$ 
value_clip: 0.2 # Prevents Critic value predictions from exploding
clip_predicted_values: True
entropy_loss_scale: 0.005 # Light entropy bonus to maintain exploratory sawing
value_loss_scale: 1.0 # Critic loss weight
grad_norm_clip: 1.0 # Max gradient norm to guard against contact force spikes
models: separate: True # Separate Actor and Critic (prevents gradient competition)
policy: class: GaussianMixin
clip_actions: True
network: - name: net
  type: linear
  layers: [256, 128, 64]
  activations: [elu, elu, elu]
value: class: DeterministicMixin
network: - name: net
  type: linear
  layers: [256, 128, 64]
  activations: [elu, elu, elu]
```

Part 7: Interactive Tablet Self-Test Quiz (Test Your Understanding)

Question 1: In PPO, what happens when an action yields a positive advantage ($A > 0$) and its probability ratio reaches $r_t(\theta) = 1.35$ (with $\epsilon = 0.2$)?

Answer: The ratio 1.35 exceeds the upper clip boundary $1 + \epsilon = 1.2$. The objective term is clipped to $1.2 \times A$, causing the gradient with respect to policy weights θ to drop to exactly zero. This prevents the optimizer from aggressively over-committing to this single action.

Question 2: Why did OpenAI abandon PPO-Penalty (adaptive KL penalty) in favor of PPO-Clip?

Answer: PPO-Penalty requires continuous manual heuristics to scale the penalty coefficient β . If β is tuned incorrectly, updates become either too conservative (freezing learning) or too aggressive (causing collapse). PPO-Clip enforces the trust region boundary directly in the objective with a fixed scalar $\epsilon = 0.2$, requiring zero hyperparameter tuning.

Question 3: Why does `value\_clip: 0.2` matter when the knife breaks through the tomato skin?

Answer: Cuticle rupture causes an abrupt drop in contact forces and a surge in rewards. Without value clipping, the Critic's loss would spike, causing a massive gradient update that distorts value predictions for all non-contact states. Value clipping restricts $V(s)$ updates to within ± 0.2 of previous estimates, preserving value stability.

Part 8: Thesis Defense Master Cheatsheet (Lecture 10 Focus)

Q1: "WHAT IS POLICY COLLAPSE, AND HOW DOES PPO PREVENT IT DURING TRAINING?"

Answer: "Policy collapse occurs in reinforcement learning when an oversized gradient step alters policy behavior such that all future rollouts result in failure. Because the policy collects its own training data, the replay buffer is corrupted with catastrophic trajectories, making recovery impossible. PPO prevents this by clipping the probability ratio $r_t(\theta) = \pi_\theta / \pi_{\text{old}}$ within $[1 - \epsilon, 1 + \epsilon]$ ($\epsilon = 0.2$). This bounds the policy update to a local trust region, mathematically preventing destructive updates."

Q2: "WHY CAN PPO PERFORM MULTIPLE TRAINING EPOCHS ON THE SAME ROLLOUT BATCH, WHILE REINFORCE CANNOT?"

Answer: "REINFORCE assumes on-policy data sampled strictly from the current policy distribution. As soon as a single gradient step is taken, the policy changes, rendering the old rollout data off-policy and invalid. PPO incorporates an importance-sampling ratio $r_t(\theta)$ combined with surrogate clipping. This enables the algorithm to safely execute 4 to 8 gradient epochs on the same rollout buffer without causing policy divergence, dramatically increasing sample efficiency."

Q3: "WHY CHOOSE PPO OVER TRPO FOR TRAINING IN ISAAC LAB?"

Answer: "TRPO strictly enforces the trust region via a hard KL constraint ($\mathbb{E}[D_{\text{KL}}] \leq \delta$), which requires computing the Fisher Information Matrix and solving a quadratic program using Conjugate Gradients. This is computationally expensive and difficult to parallelize on GPUs. PPO achieves equivalent empirical stability through a simple first-order clipped surrogate objective that integrates seamlessly with standard GPU optimizers like Adam."